Informed Search

Kaiqi Zhao
Harbin Institute of Technology, Shenzhen

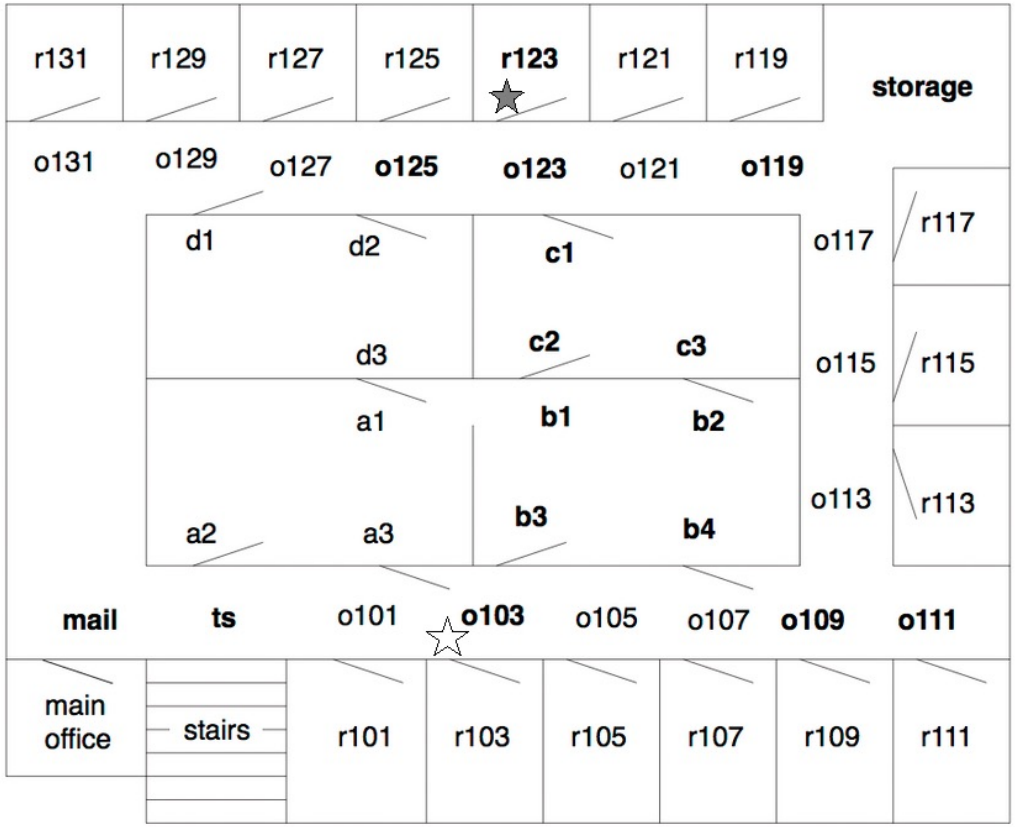
Informed Search Strategies (Heuristic Search)

A heuristic refers to a rule or a piece of information to facilitate problem solving; a heuristic will enable a solution to be easily or efficiently found, but the found solution does not guaranteed to be optimal, perfect, logical, or rational.

Definition

A search heuristic function $h(u)$ takes a node u and returns a non-negative real number that is an estimate of the cost of the least-cost path from u to a goal node.

Example. [Route planning of a robot] The only way a robot can get through a doorway is to push the door open in the direction shown. Suppose the robot starts from **o103**, and aims to get to **r123**.



Heuristic: the straight-line distance between a location and the goal

$$
\begin{array}{rrrrrr}
h(mail) = 26 & h(ts) = 23 & h(o103) = 21 \\
h(o109) = 24 & h(o111) = 27 & h(o119) = 11 \\
h(o123) = 4 & h(o125) = 6 & h(r123) = 0 \\
h(b1) = 13 & h(b2) = 15 & h(b3) = 17 \\
h(b4) = 18 & h(c1) = 6 & h(c2) = 10 \\
h(c3) = 12 & h(storage) = 12 &
\end{array}
$$

Recall: Generic Search Scheme

Generic Search(G, s, F)

INPUT: Graph $G = (S, E)$, start node $s \in S$, goal nodes $F \subseteq S$
OUTPUT: a path from s to some node in F if exists; otherwise $\perp$

Initialise a collection Frontier $\leftarrow \{(s)\}$
while Frontier $\neq \emptyset$ **do**
Select and remove a path $(s, \dots, u)$ from Frontier
if $u \in F$ **then**
return $(s, \dots, u)$
end if
Expand $(s, \dots, u)$ with successors v and insert the new paths into Frontier
end while
return $\perp$

Informed Search 1: Greedy Best First Search (GBFS)

GBFS(G, s, F)

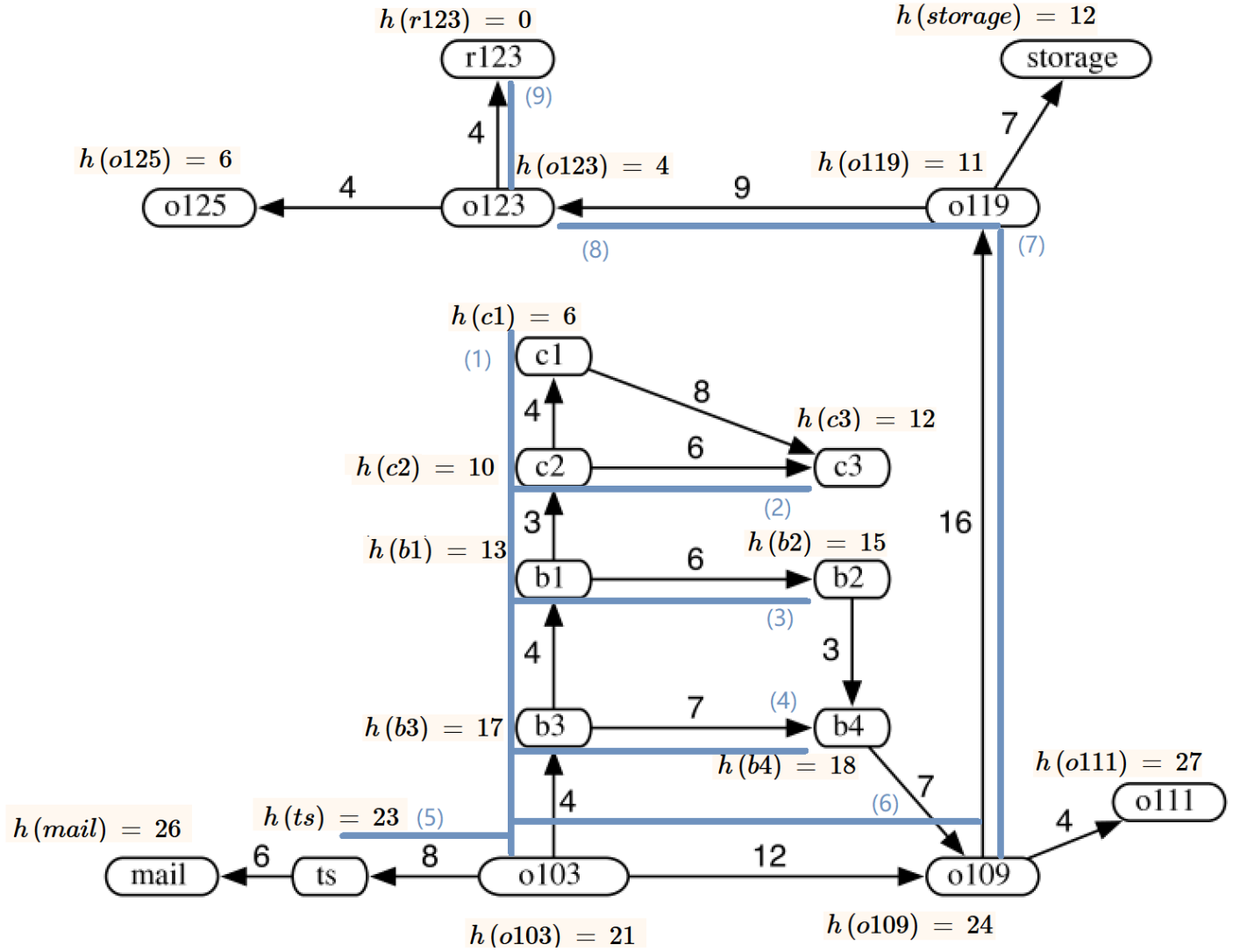
INPUT: Graph $G = (S, E)$, start node $s \in S$, goal nodes $F \subseteq S$
OUTPUT: a path from s to some node in F if exists; otherwise $\perp$

Initialise a priority queue Frontier $\leftarrow \{((s), h(s))\}$
while Frontier $\neq \emptyset$ **do**
  Remove the top path $(s, \dots, u)$ from Frontier with smallest $h(u)$
  if $u \in F$ **then**
    return $(s, \dots, u)$
  end if
  Expand $(s, \dots, u)$ with successors v and insert the new paths with value $h(v)$ into Frontier
end while
return $\perp$

Example. [Route planning of a robot]

Heuristic: the straight-line distance between a location and the goal

$$
\begin{array}{lll}
h(mail) = 26 & h(ts) = 23 & h(o103) = 21 \\
h(o109) = 24 & h(o111) = 27 & h(o119) = 11 \\
h(o123) = 4 & h(o125) = 6 & h(r123) = 0 \\
h(b1) = 13 & h(b2) = 15 & h(b3) = 17 \\
h(b4) = 18 & h(c1) = 6 & h(c2) = 10 \\
h(c3) = 12 & h(storage) = 12 &
\end{array}
$$



Informed Search 2: A* Search

A* search uses an estimate of the total cost from the start node to a goal node via a path.

- $c(u)$ represents the cost of going from s to u via the shortest expanded path p
- $h(u)$ represents the estimated cost from the node u to a goal in F.
- Define $f(u) := c(u) + h(u)$ as the estimate of the total cost to follow p then go to the goal node.
- A* search uses a priority queue ordered by $f(u)$

A* Search(G, s, F, h)

INPUT: Graph $G = (S, E)$, start node $s \in S$, goal nodes $F \subseteq S$, heuristic h, edge weight w: $E \rightarrow \mathbb{R}$ denoting edge costs

OUTPUT: a path from s to some node in F if exists; otherwise $\perp$

Create a priority queue Frontier containing $((s), f(s) = h(s))$

while Frontier $\neq \emptyset$ **do**

Select the path $(s, \ldots, u)$ from Frontier that has the minimum $f(s)$

if $u \in F$ **then**

return $(s, \ldots, u)$

end if

for $v \in V$ such that $(u, v) \in E$ **do**

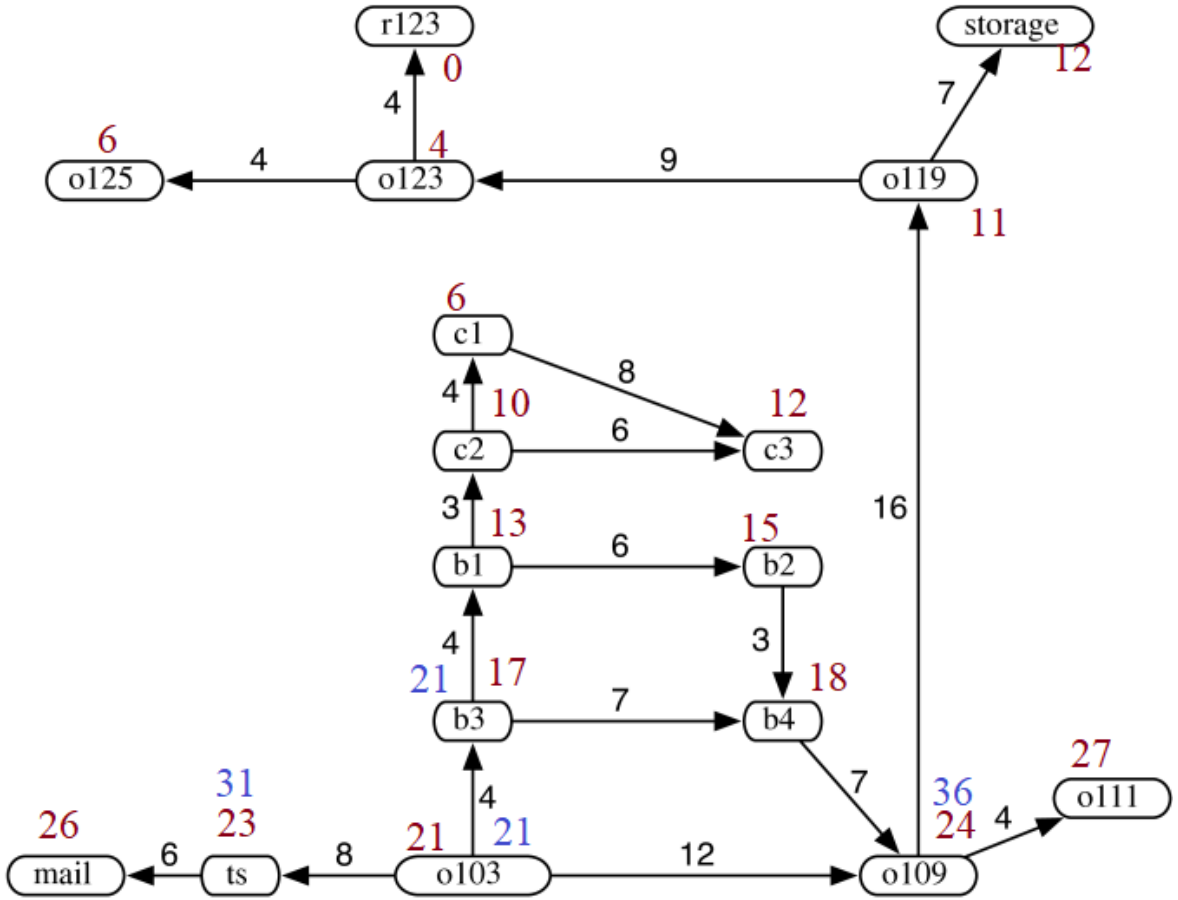
If $c(u) + w(u, v) < c(v)$, then set $c(v) \leftarrow c(u) + w(u, v)$ and add $(s, \ldots, u, v)$ into Frontier with value $f(v) = c(v) + h(v)$

end for

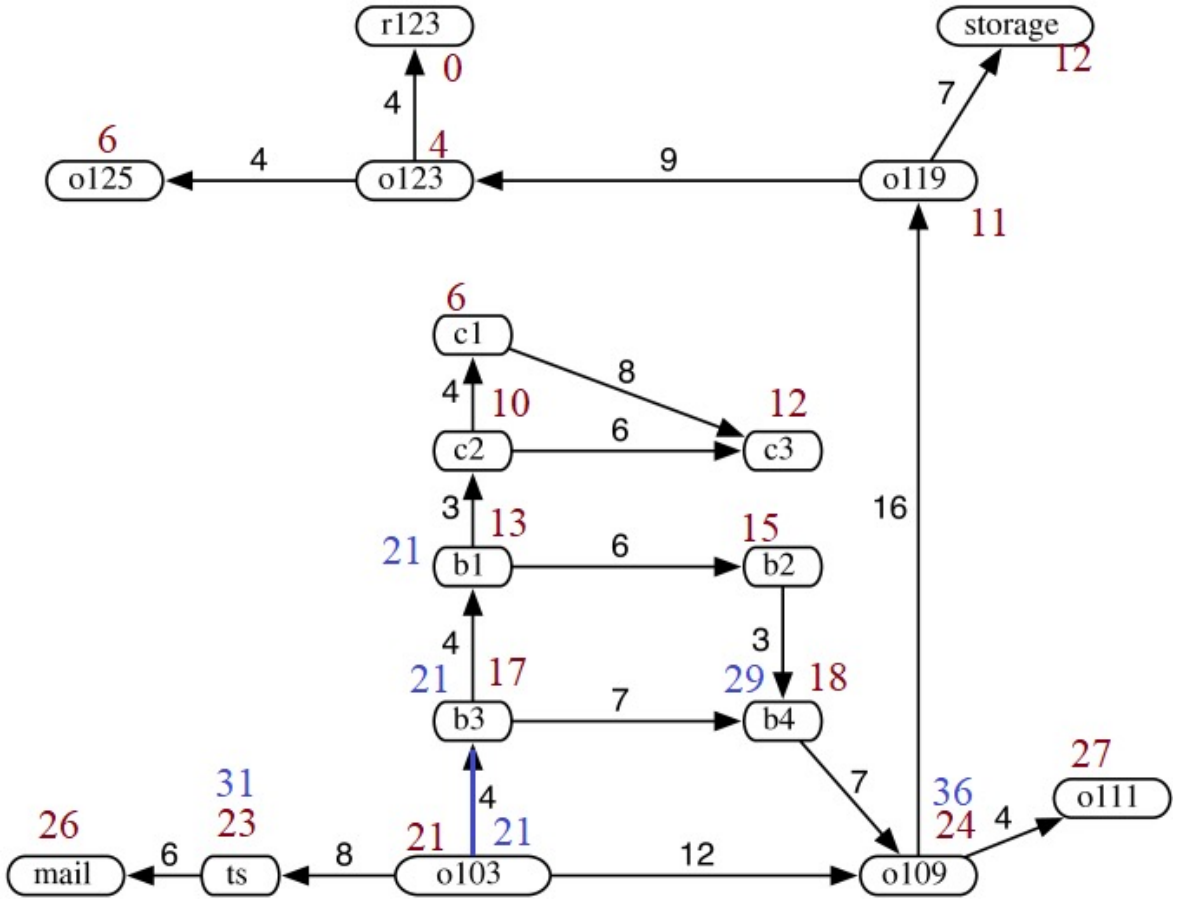
end while

return $\perp$

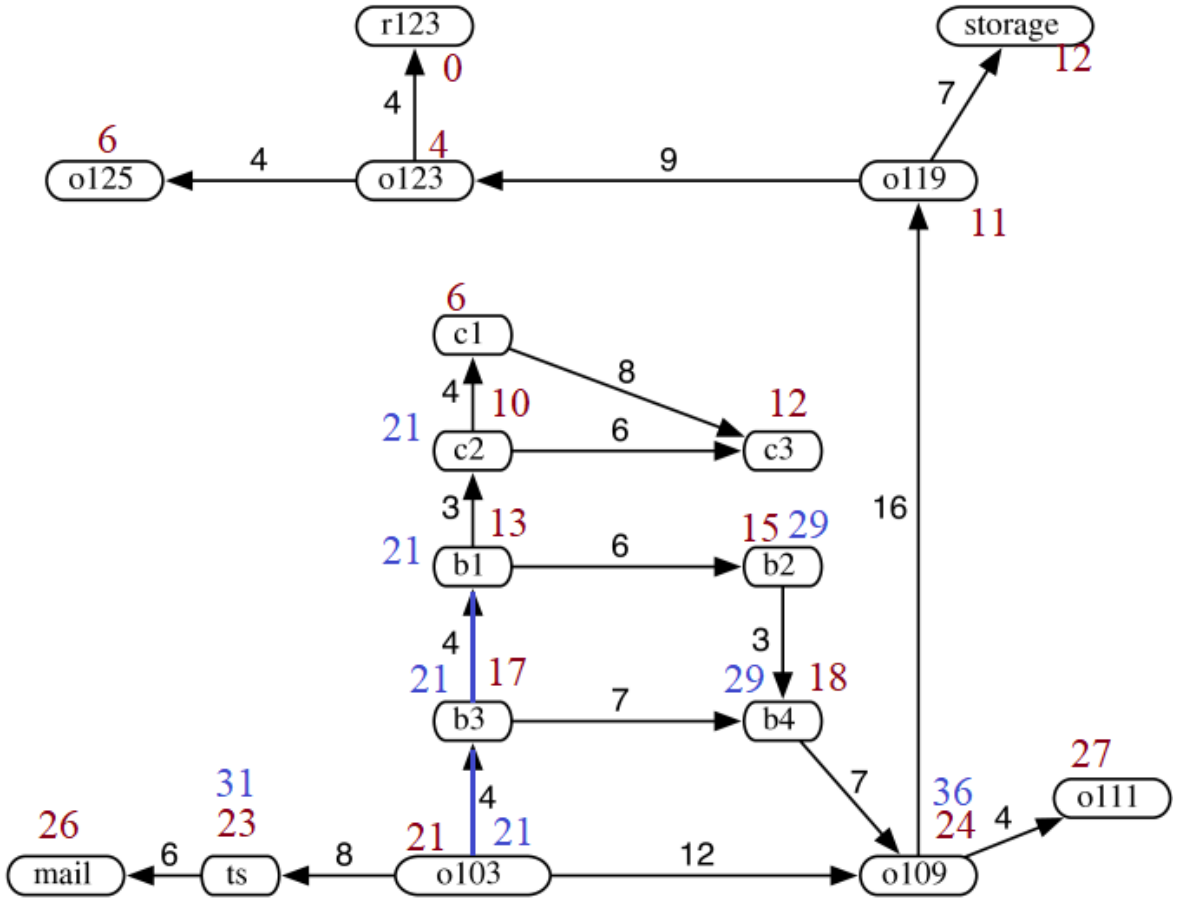
Example. [Route planning of a robot] Apply A* search with the given heuristic



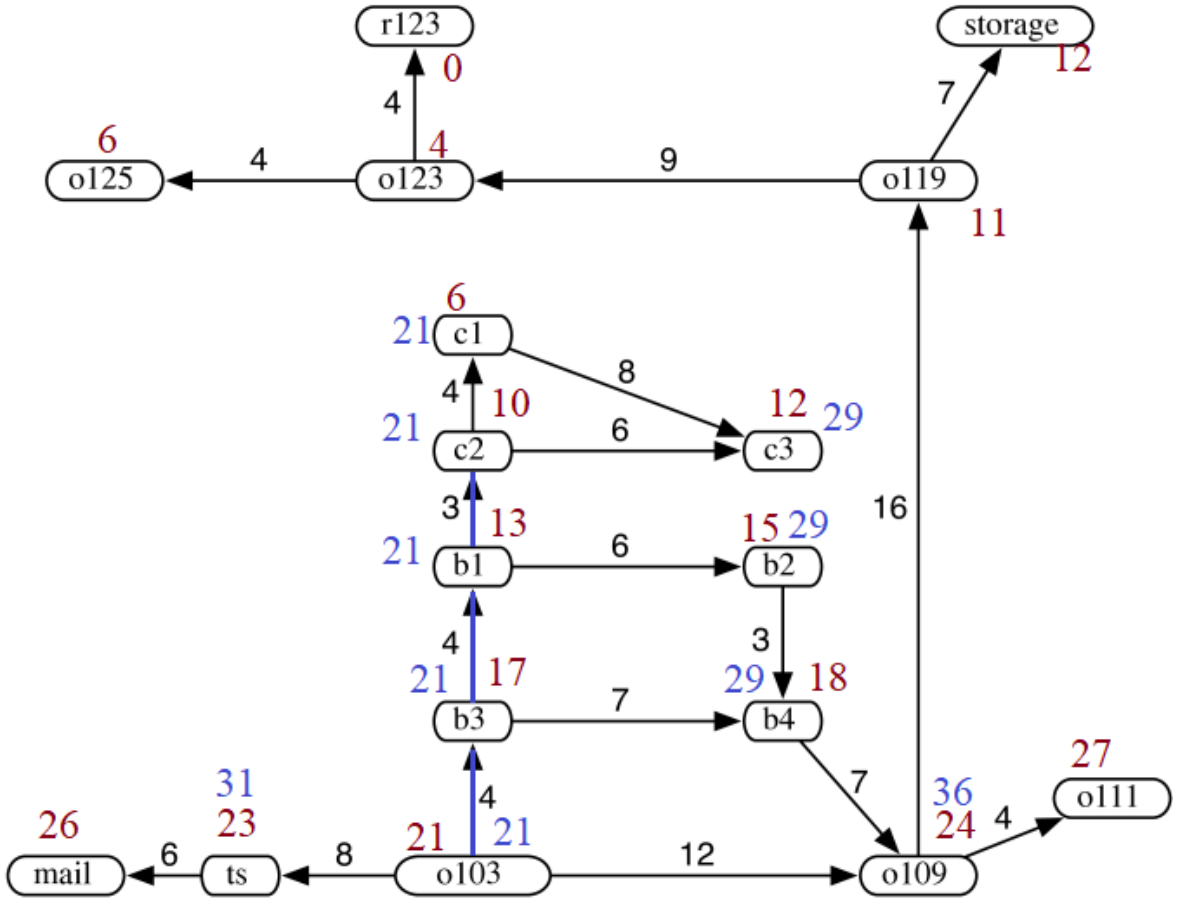
Example. [Route planning of a robot] Apply A* search with the given heuristic



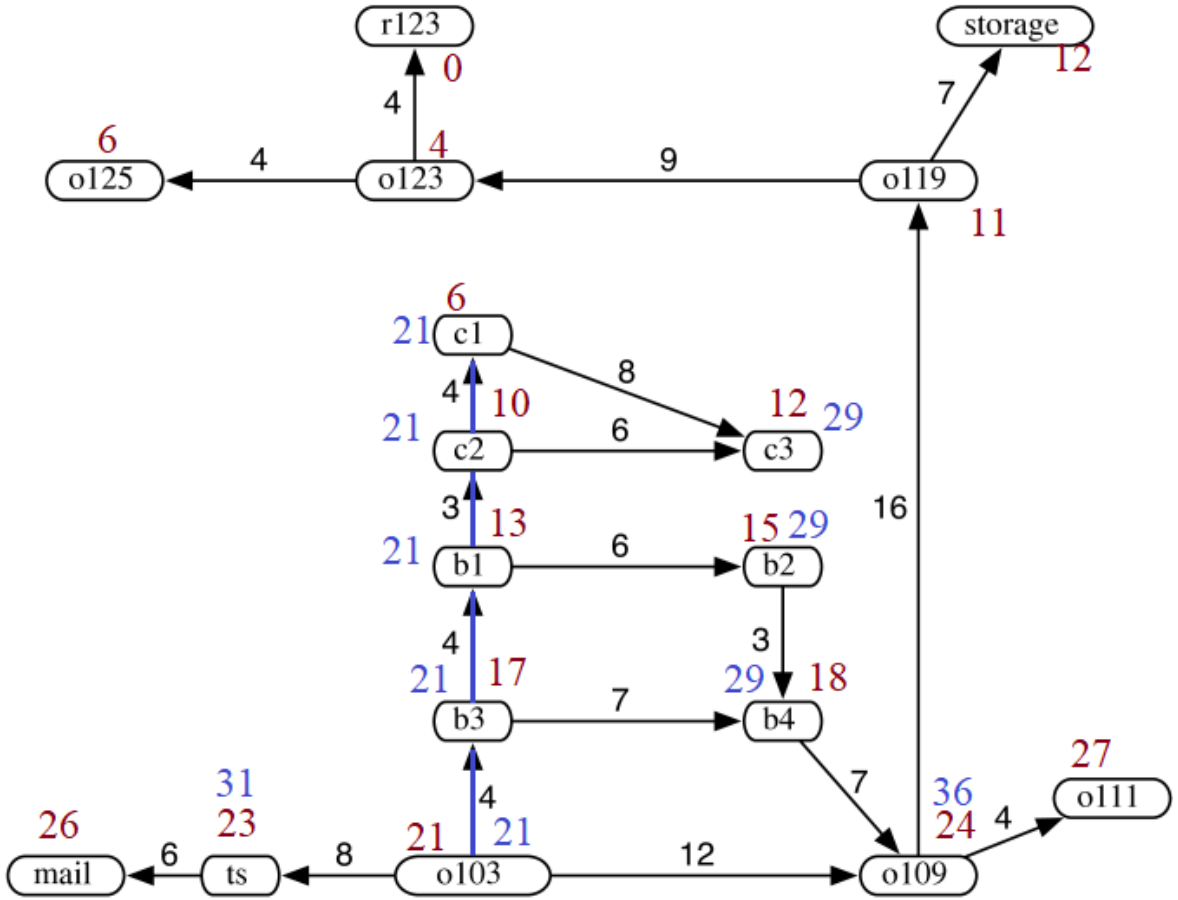
Example. [Route planning of a robot] Apply A* search with the given heuristic



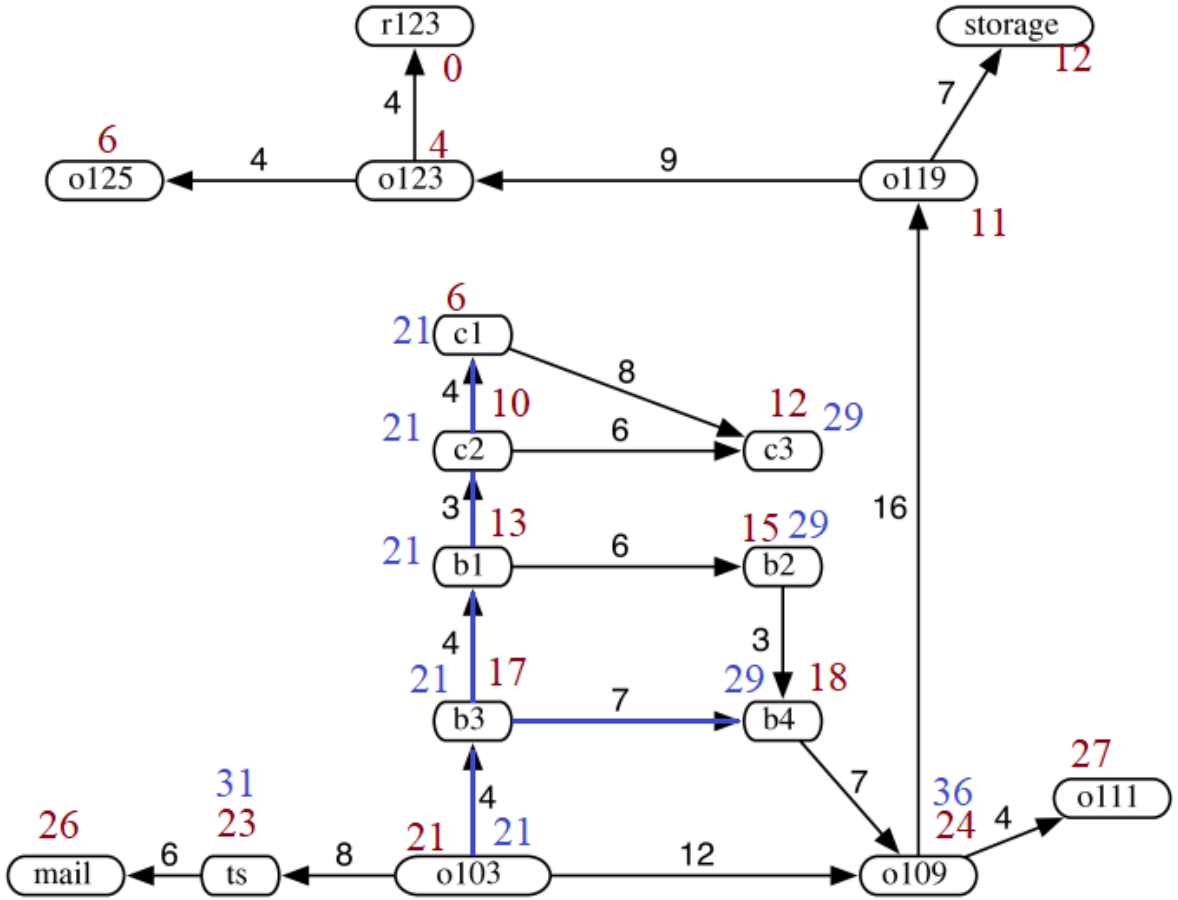
Example. [Route planning of a robot] Apply A* search with the given heuristic



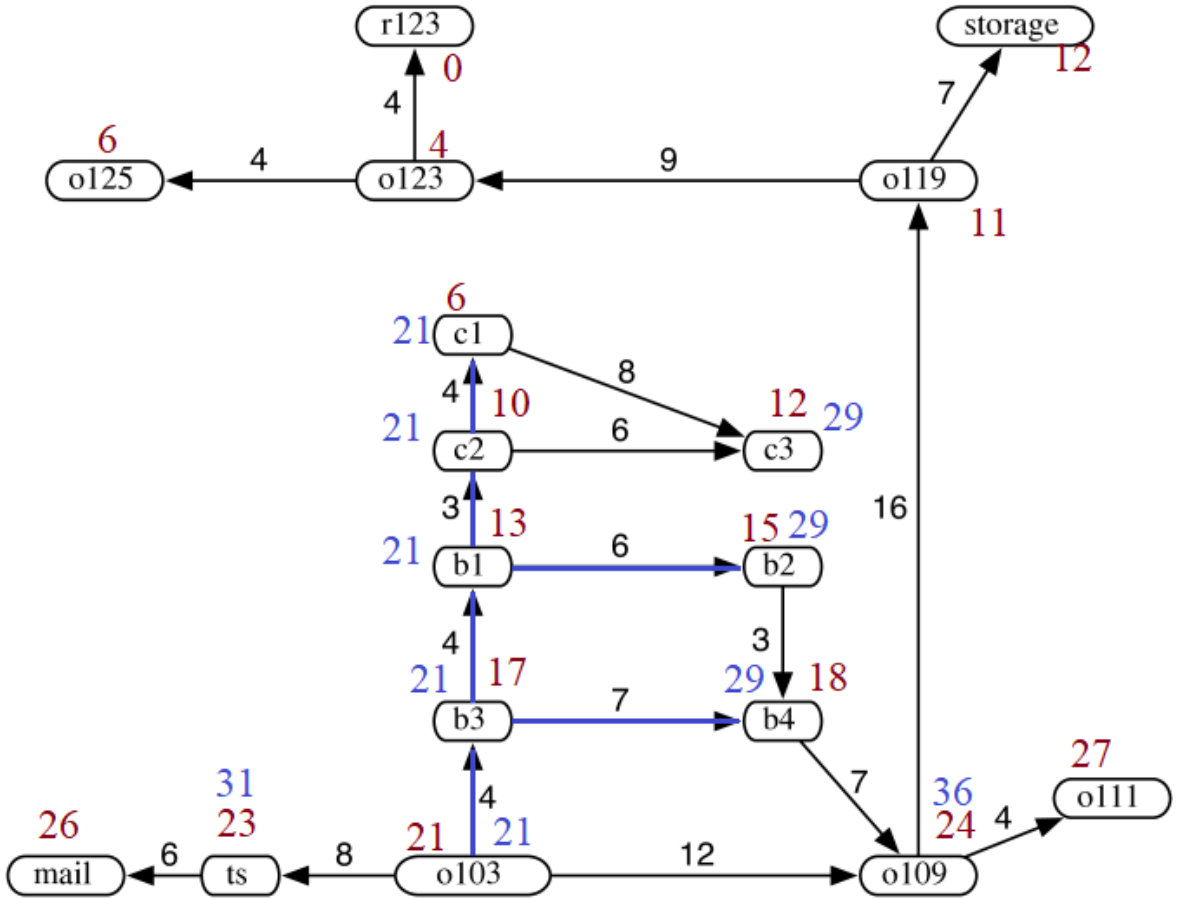
Example. [Route planning of a robot] Apply A* search with the given heuristic



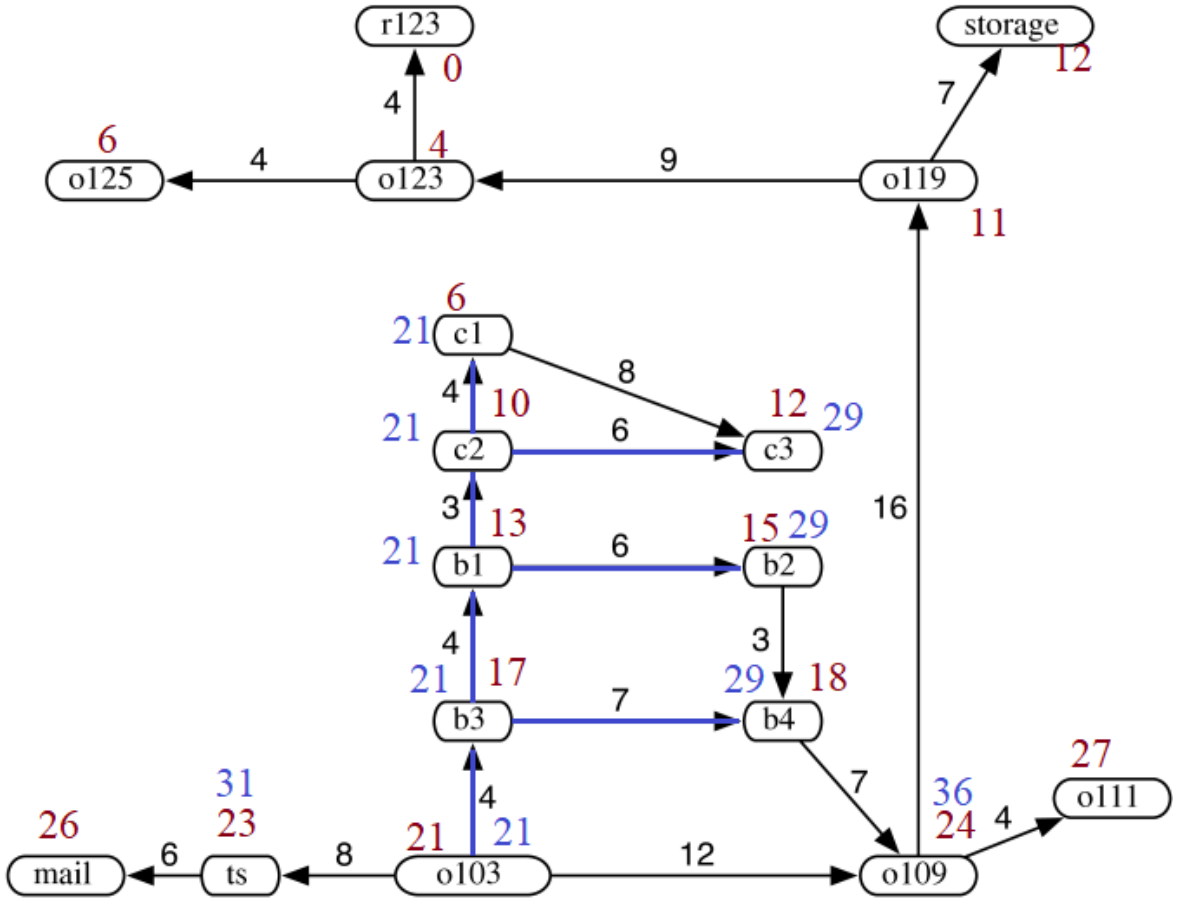
Example. [Route planning of a robot] Apply A* search with the given heuristic



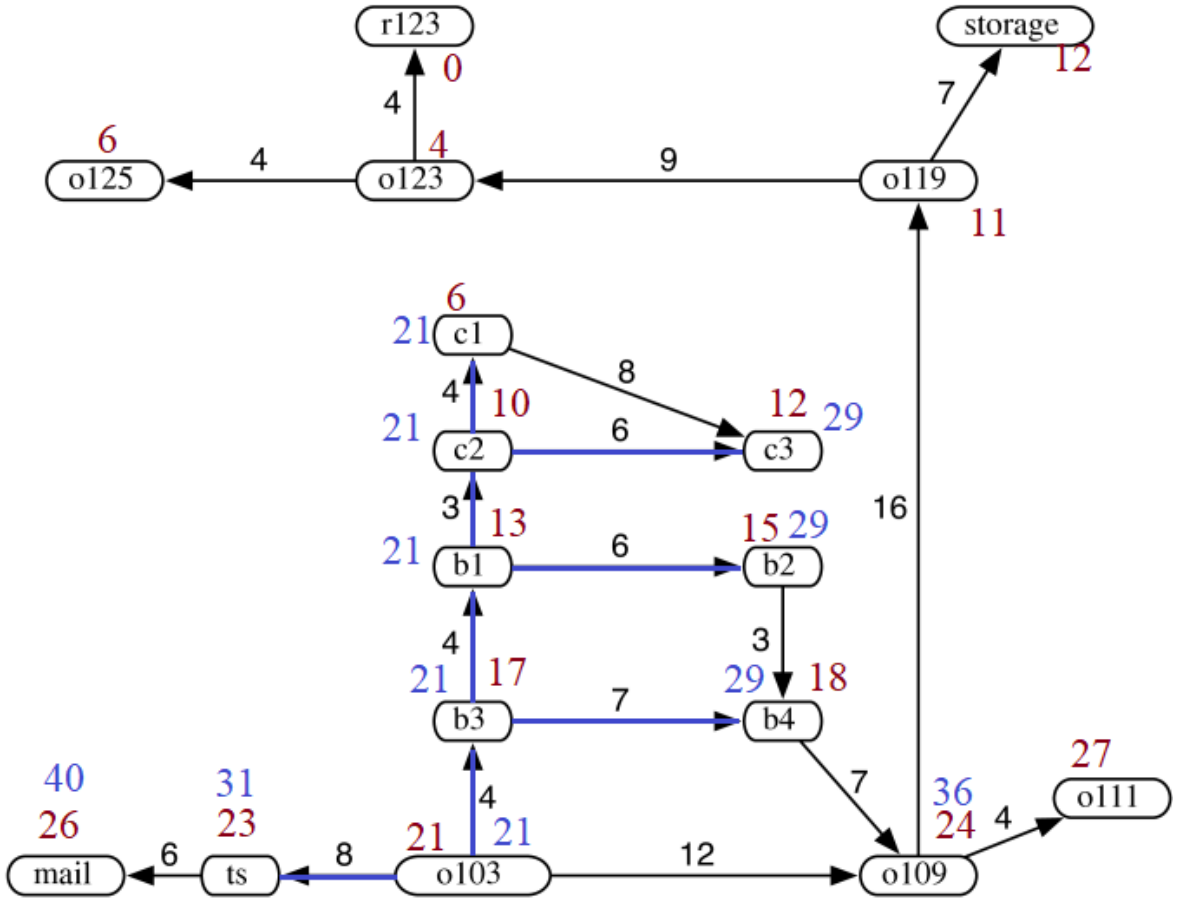
Example. [Route planning of a robot] Apply A* search with the given heuristic



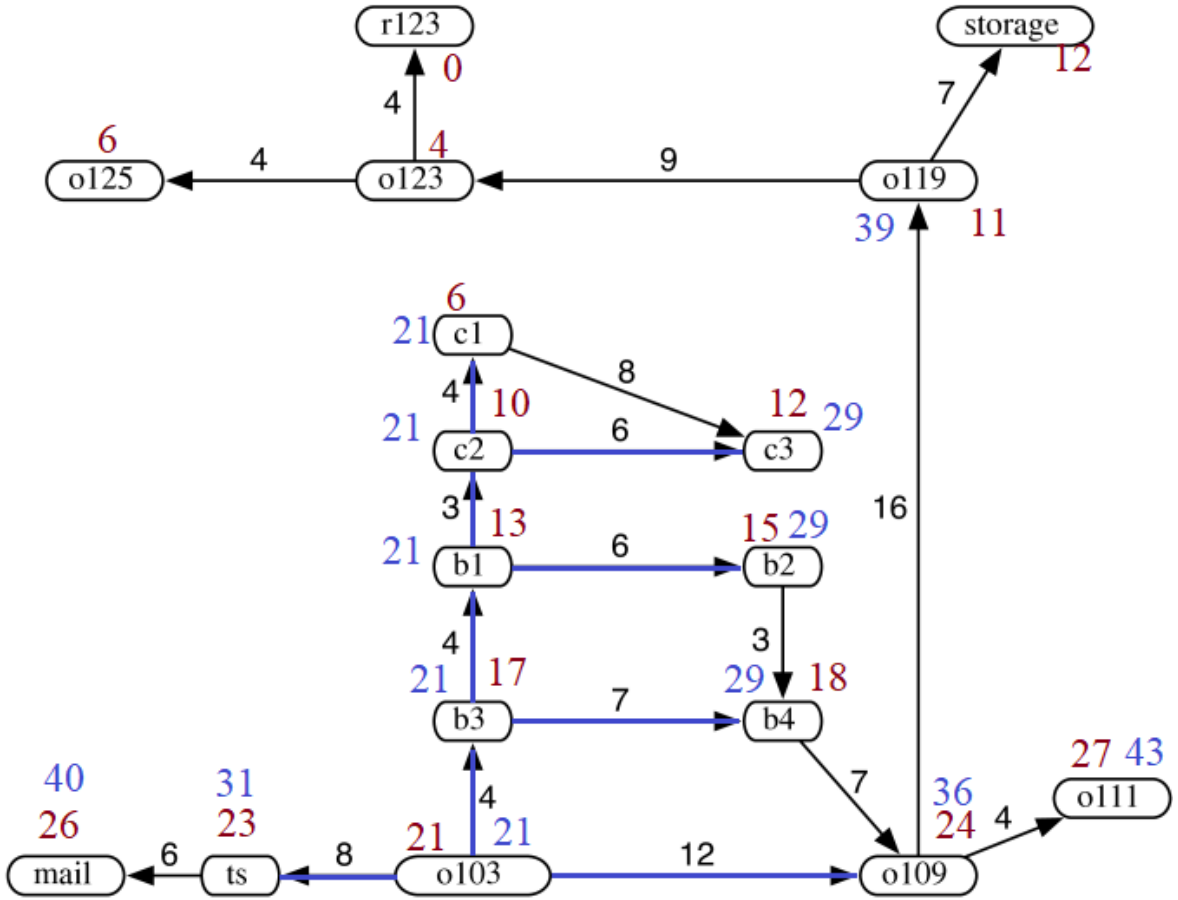
Example. [Route planning of a robot] Apply A* search with the given heuristic



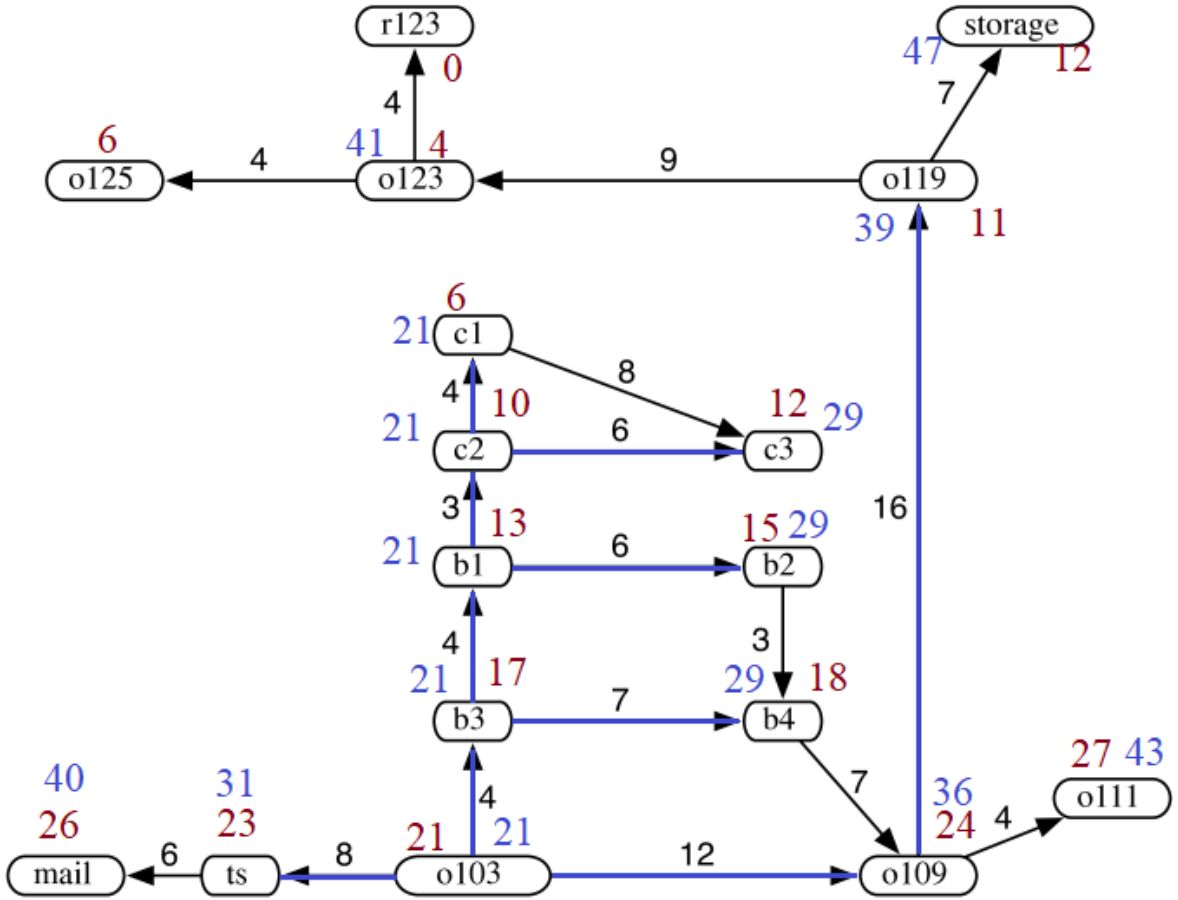
Example. [Route planning of a robot] Apply A* search with the given heuristic



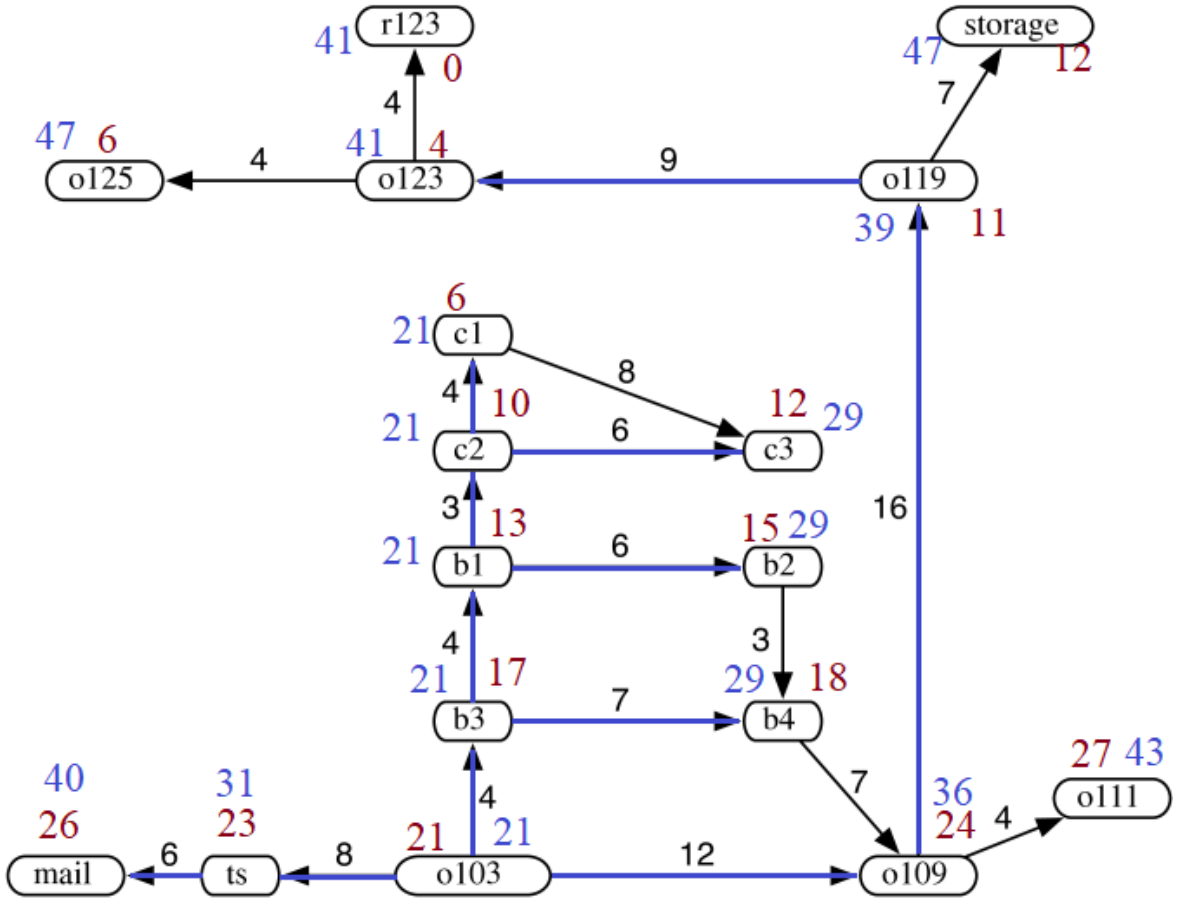
Example. [Route planning of a robot] Apply A* search with the given heuristic



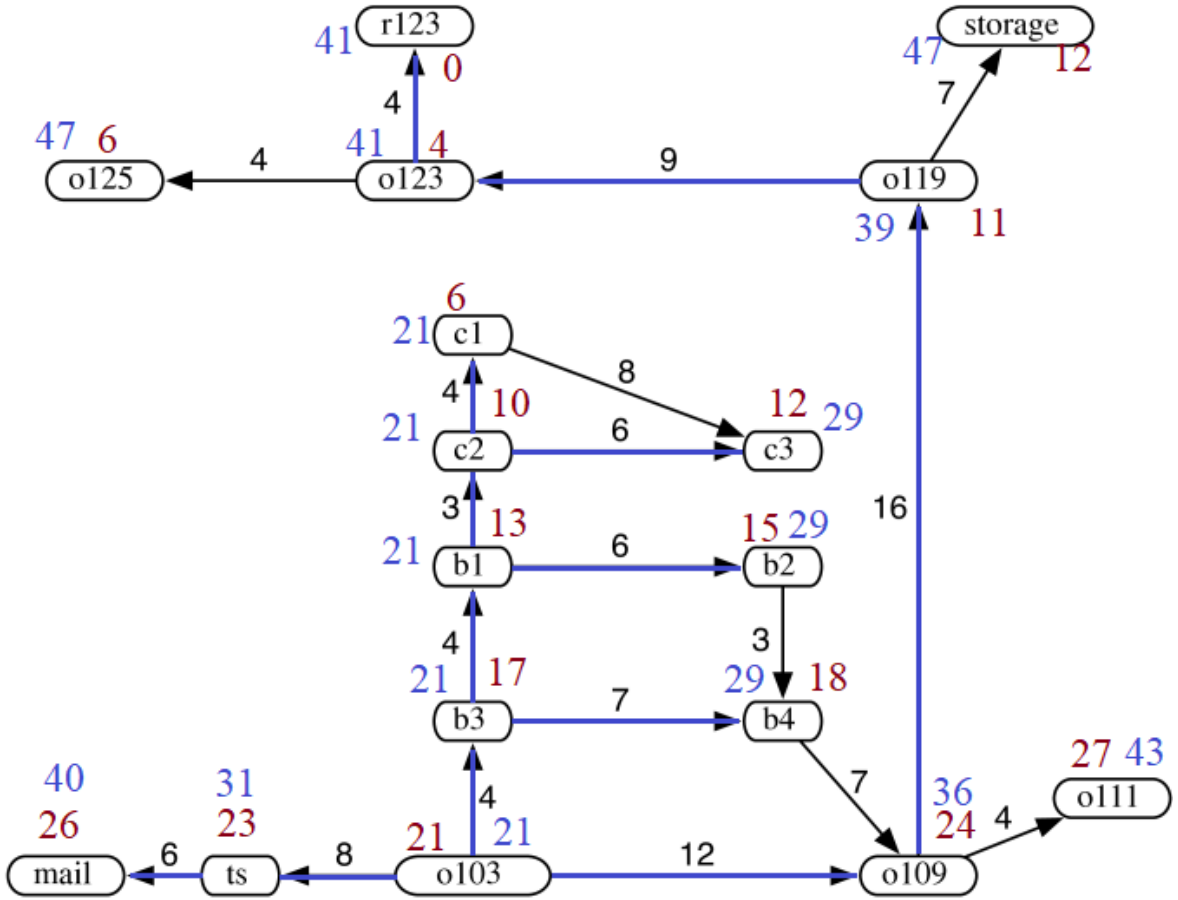
Example. [Route planning of a robot] Apply A* search with the given heuristic



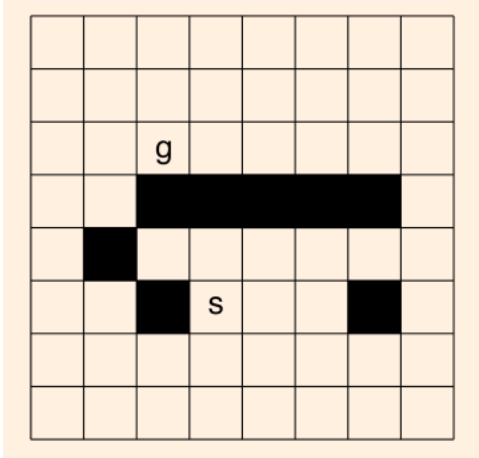
Example. [Route planning of a robot] Apply A* search with the given heuristic



Example. [Route planning of a robot] Apply A* search with the given heuristic



Example. Consider the grid problem. Use A^* algorithm to solve this problem.



Apply this method to the problem assuming $h(x)$ of a given cell in the grid equals to the Manhattan distance from x to g.
Assume we apply multiple-path pruning. Tie breaks are handled by the order up > left > right > down.

Formal analysis of A* Search

A search heuristic function $h(u)$ is admissible if it does not over-estimate the cost for any node, i.e., for any node $u \in S$:

$$h(u) \leq h^*(u)$$

where $h^*(u)$ is the cheapest cost of any path going from u to a node in F

Example. [Route planning of a robot] The following heuristics are admissible

- $h(u) = 0$ for every node $u \in S$
- $h(u) = 1$ for every node $u \notin F$ and $h(u) = 0$ for every node $u \in F$
- $h(u) =$ the smallest straight-line distance to any goal node in F

A* Admissibility Theorem

Suppose the graph G is finite and there is a path from the start node to a goal node.

If A* search uses an admissible heuristic function, then it always returns an optimal solution.

Proof. Note that a solution must be returned as the search algorithm explores all nodes reachable from the start node via a path.

- **Idea**: Suppose $(u_1, \dots, u_k)$ be the path with the optimal cost C^* and the path $(v_1, \dots, v_m)$ found by A* is not optimal. We can proof by contradiction.
  - Note that $u_1 = v_1 = s$, but u_k may not be the same node as v_m

- For any $i \in \{1, \dots, k\}$

$$f(u_i) = c(u_i) + h(u_i) \leq c(u_i) + h^*(u_i) = f(u_k) \leq f(v_m).$$

- Let i be the least index where u_i is not expanded by A*.
- Since A* always expands the node with the least f, before expanding v_m, it will expand u_i. Contradiction.

Question. How does A^* improve efficiency of search?

- The efficiency of A^* relies on how many paths are expanded from Frontier during the computation.
- The number of expanded paths depends on the heuristic h.
- Suppose h is admissible. Then A^* expands all paths from the start node of the form

  $(s, \ldots, u)$ where $\mathrm{c}(u) + h(u) < C^*$

  and some paths of the form

  $(s, \ldots, u)$ where $\mathrm{c}(u) + h(u) = C^*$

- Increasing h while keeping it admissible reduces the first type of paths expansions.

Informedness of Heuristics

Definition

A heuristic h_1 is said to be more informed than h_2 if both are admissible and

$$h_1(u) > h_2(u) \text{ for any node } u \notin F$$

Example. [Route planning of a robot] The following heuristics are admissible

- $h_1(u) =$ the smallest straight-line distance to any goal node in F
- $h_2(u) =$ half of the smallest straight-line distance to any goal node in F
- h_1 is more informed than h_2

Question: How does the informedness influence the efficiency of A* search?

Heuristic Comparison Theorem

Suppose A_1, A_2 are A* strategies with admissible heuristics h_1, h_2, respectively.

If h_2 is more informed than h_1, then every path expanded by A_2 is also expanded by A_1

Proof.

- If A_2 expands a path $p = (u_1, \dots, u_k)$, then $c(u_k) + h_2(u_k) \leq C^*$
- For any prefix path of p, e.g., $(u_1, \dots, u_i)$, if h_1 is applied, $c(u_i) + h_1(u_i) < c(u_i) + h_2(u_i) \leq C^*$
- Because
  - A* search always prioritizes the expansion of path with the smallest f, and
  - A* search with admissible heuristics always find the optimal solution (by Admissibility theorem)

  All these prefix paths with $f(u_i) = c(u_i) + h_1(u_i) < C^*$ will eventually be expanded before A_1 expands the optimal path with cost C^*.
- Therefore, any path expanded by A_2 is also expanded by A_1

Conclusion: A more informed heuristic can make A* more efficient

Summary of Search Strategies

The table summarises the search strategies over finite graphs where edge costs are positive:

Strategy	Selection rule from Frontier	Path found	Space
DFS	Last path added	No guarantee[1]	$O(bm)$
BFS	First path added	Fewest edges	$O(b^k)$
UCS	Minimal cost	Least cost	$O(b^k)$
ID	Depth-bounded	Fewest edges	$O(bk)$
GBFS	Minimal h(p)	Sub-optimal	$O(b^k)$
A*	Minimal c(p) + h(p)	Least cost[2]	$O(b^k)$

All these search strategies have worst-case time complexity exponential w.r.t. the number of edges in the paths.

[1]Not guarantee to terminate if without pruning
[2]Require admissible search heuristic *h*

More on Informed Search

Weighted A* Search

Rethinking the evaluation function $f = c + h$ of A* search

- c is to make sure that we pursue the cheapest path; so that we are not trapped into searching along sub-optimal paths.
- h is to speed up the search by moving along the direction that is most likely optimal.

Two extremes:

- $f = c$ (UCS): Optimal solution is guaranteed to be found, but normally not efficient.
- $f = h$ (GBFS): Potentially sub-optimal solution may be found, but normally efficient.

Weighted A* Search

We can add weight $w \geq 0$ to balance c and h:

$$f_w(u) = c(s, \ldots, u) + wh(u)$$

- $w = 1$: A* search
- $w < 1$: Still produce optimal solution, with slower speed (less informed heuristic)
- $w > 1$: Higher speed, but may produce sub-optimal solutions

The w-weigthed A* search strategy is the same as A* search except that f_w replaces f.

Question: How bad would the sub-optimal solution be, if we set $w > 1$?

Consider a time before the termination of the w-weighted A*:

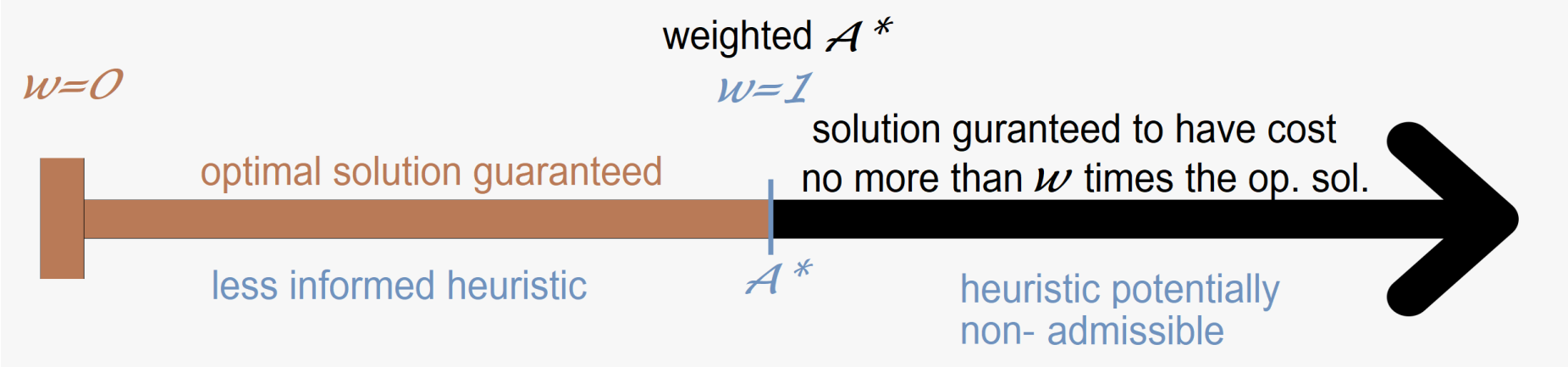
- Take an optimal path P^*.
- Let $(s, \dots, u)$ be the shortest path in Frontier along P^*
- When the algorithm expands from u, $c(u)$ must store the optimal cost from s to u, i.e., $c(u) = c^*(u)$

$$
\begin{aligned}
f_w(u) &= c(u) + wh(u) \\
&= c^*(u) + h(u) + (w-1)h(u) \\
&\le c^*(u) + h^*(u) + (w-1)h^*(u) && \text{(By Admissibility)} \\
&= C^* + (w-1)h^*(u) \\
&\le wC^* && (h^*(u) \le C^*)
\end{aligned}
$$

Definition

A search strategy is ϵ-admissible if it guarantees to find a path with cost $\leq (1+\epsilon)C^*$ and $\epsilon > 0$

A w-weighted A* search strategy where $w > 1$ is $(w-1)$-admissible



IDA*: Iterative Deepening A*

A* may consume a large space as Frontier can grow exponentially with respect to number of edges in the paths.

- IDA* performs repeated cost-bounded DFS while setting the cost bound on the value of $f(n)$.
- Initial cost bound is $f(s)$ where s is the start node.
- Carry out DFS but never expands a path with a higher f–value than the current bound.
- If the DFS does not find a solution, increase the bound by δ.

IDA* can improve the empirical efficiency of search.

Smart Pruning: Depth-first Branch-and-bound Search

Our aim
1. Guarantee to find the optimal path.
2. Use linear space cost.
3. Prune search space to make the search fast.

The depth-first branch-and-bound search applies DFS, but with the following differences:

- It maintains the lowest-cost path to a goal found so far, referring to its cost as bound.
- When encountering a path $p = u_1, u_2, \dots, u_k, v$ with

$$cost(p) + h(v) \geq bound,$$

then prune this path.
- If a non-pruned path to a goal is found, it must be better than the previous best path. Then replace the previously remembered path by this path.

cbsearch$((u_1, \ldots, u_k), p, \beta, \mathrm{G}, \mathrm{F}, h)$

INPUT: path $(u_1, \ldots, u_k)$, current best path p, current bound β, graph G, heuristic h, goal F

OUTPUT: path p

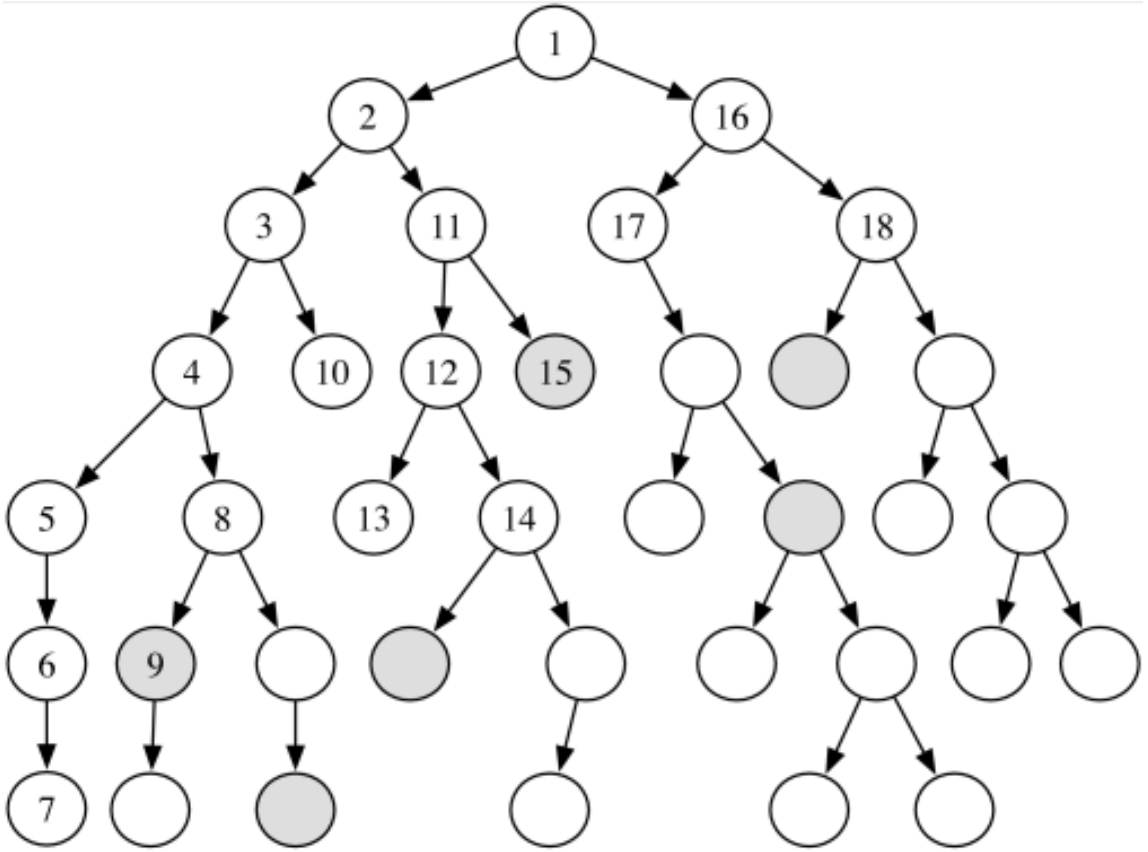
if $\mathrm{cost}(p) + h(u_k) < \beta$ **then**
  if $u_k \in F$ **then**
    Set $p \leftarrow (u_1, \ldots, u_k)$ and $\beta \leftarrow \mathrm{cost}(p)$
  else
    for edge $(u_k, v) \in E$ **do**
      $p \leftarrow cbsearch((u_1, \ldots, u_k, v), p, \beta, \mathrm{G}, \mathrm{F}, h)$
    end for
  end if
end if
return p

DF-Branch-and-Bound (G, s, F, h, β_0)

INPUT: Graph $G = (S, E)$, start node $s \in S$, goal nodes $F \subseteq S$, heuristic h,
OUTPUT: a path from s to some node in F if exists; otherwise $\perp$

return cbsearch$((s), \perp, \infty, \mathrm{G}, F, h)$

Example. DF Branch-and-bound Search. The shaded nodes are goal nodes. Edge cost is 1. $h(u) = 0$ for all $u \in S$.



Note. DF branch-and-bound search guarantees to terminate and finds an optimal path in linear space.

Summary of The Topic

The following are the main knowledge points covered:

- What is an agent system?
- What is a state-based model? What does the search problem ask for?
- What is a search strategy?
- Describe uninformed search strategies: DFS, BFS, UCS, ID.
- Describe informed search strategies (heuristics): A*, IDA*, DF Branch-and-bound.
- Discuss the computational complexity of the search algorithms.
- Discuss other search optimization techniques.